Dylan Francis DAAN 682
22MAR2026

Homework_9:

Code:

```
#
========S2==========================================================================
=
# DAAN: 682: Data Analytics Programming in Python
# Author: Dylan Francis
# Title: Homework_9: Supervised Learning with Scikit-learn II: regression Models
#
==============================================================================
#
#
==============================================================================
#
# Perform exploratory data analysis and interpret the results. Remove the 'motor_UPDRS'
column as it should not be used as a predictor. (10 points)
# Use cross-validation to build a Linear Regression model to predict total_UPDRS. Report and
analyze the results using different metrics such as MAE, MSE, and R2. (25 points)
# Use cross-validation to build a Regression Tree model to predict total_UPDRS. Report and
analyze the results using different metrics such as MAE, MSE, and R2. (25 points)
# Use cross-validation to build a Neural Network model to predict total_UPDRS. Report and
analyze the results using different metrics such as MAE, MSE, and R2. (25 points)
# Compare their performance with MAE and MSE, which model has better performance?
Propose at least one specific technique to potentially improve the performance. (5 points)
# Choose either the Regression Tree model or Neural Network model and optimize it by tuning
its parameters. Report and analyze how these parameters help improve the performance. (10
points)
#
==============================================================================

import os
import pandas as pd
import numpy as np
from sklearn import datasets
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import KFold, cross_validate
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```
path =
r"C:\Users\dylan\OneDrive\Documents\GRAD_SCHOOL\DAAN_682\HOMEWORK_9\parkinsons
+telemonitoring"
os.chdir(path)
df = pd.read_csv("parkinsons_updrs.data")


#1.) Exploratory analysis and interpreting results.
print(f"\nThere are {df.shape[0]} rows, and {df.shape[1]} columns in the features dataframe")
print("\n=======DATASET_INFO======")
print(df.info())
print("\n=======SUMMARY_STATS======")
print(df.describe())
print("\n===== MISSING VALUES =====")
print(f"{df.isnull().sum()}")
print("\n===== DUPLICATE ROWS =====")
print(f"The total number of duplicate rows is: {df.duplicated().sum()}")

df_modified = df.drop(["motor_UPDRS", "subject#"], axis=1)
print(df_modified)
print(f"\nThere are {df_modified.shape[0]} rows, and {df_modified.shape[1]} columns in the
features dataframe")
print("\n=======DATASET_INFO======")
print(df_modified.info())
print("\n=======SUMMARY_STATS======")
print(df_modified.describe())
print("\n===== MISSING VALUES =====")
print(f"{df_modified.isnull().sum()}")

plt.figure(figsize=(12, 10))
df_corr = df_modified.corr()
sns.heatmap(df_corr, cmap='coolwarm', center=0)
plt.title("Correlation Matrix")
plt.show()

# Targets
fig, ax = plt.subplots(figsize=(12, 5))
sns.histplot(df_modified['total_UPDRS'], kde=True, ax=ax)
plt.show()
print("Based on the histogram plot of the target plot, the data is approximately normal,
potentiallly slight right skew.")
print("if the data is right skewed, then that would suggest most patients have lower UPDs
scores, with fewer high-severity cases.")
```

```
corr = df_modified.corr()['total_UPDRS'].sort_values()
corr.plot(kind='barh', figsize=(8,10))
plt.title("Feature Correlation with total_UPDRS")
plt.show()
print("This grahph shows the features displayed from most positvely correlated to most
negatively correlated.")

#2.) getting into the actaul modeling now (regression)
kf = KFold(n_splits=5, shuffle=True, random_state=42)

x = df_modified.drop(columns=['total_UPDRS'])
y = df['total_UPDRS']

lin_reg_model = LinearRegression()
scoring = {'MAE': 'neg_mean_absolute_error', 'MSE': 'neg_mean_squared_error', 'R2': 'r2'}

results = cross_validate(lin_reg_model, x, y, cv=kf, scoring=scoring)

# Convert negative errors to positive (since higher is "better")
lr_mae = -results['test_MAE']
lr_mse = -results['test_MSE']
lr_r2 = results['test_R2']

print("\n \n Linear Regression Results ")
print("MAE:", lr_mae)
print("MSE:", lr_mse)
print("R2:", lr_r2)

print("\n Here are the average results for Linear Regression")
print("Mean MAE:", np.mean(lr_mae))
print("Mean MSE:", np.mean(lr_mse))
print("Mean R2:", np.mean(lr_r2))

print(f"\n The mean MAE is {np.mean(lr_mae)}, which is a fairly large error, suggesting low
prediction accuracy.")
print(f"\n The mean MSE is {np.mean(lr_mse)}, which means the model is making large
prediction errors, suggesting some outliers could exisit.")
print(f"\n A R^2 of {np.mean(lr_r2)} suggests only  {np.mean(lr_r2)*100} % of the total variance
of the target is predicted by the model.")

#3.) Decision tree:
tree_model = DecisionTreeRegressor(random_state=42)
```

```
tree_results = cross_validate(tree_model, x, y, cv=kf, scoring=scoring)

# Convert negative errors to positive (since higher is "better")
tree_mae = -tree_results['test_MAE']
tree_mse = -tree_results['test_MSE']
tree_r2 = tree_results['test_R2']

print("\n \n Regression Tree Results ")
print("MAE:", tree_mae)
print("MSE:", tree_mse)
print("R2:", tree_r2)

print("\n Average Results (Regression Tree)")
print("Mean MAE:", np.mean(tree_mae))
print("Mean MSE:", np.mean(tree_mse))
print("Mean R2:", np.mean(tree_r2))

print(f"\n The mean MAE is {np.mean(tree_mae)}, which is a very small error, suggesting high
prediction accuracy. This is predicting UPDRS within 1 unit")
print(f"\n The mean MSE is {np.mean(tree_mse)}, which is pretty small, suggesting very few
extreme outliers. There is strong consistenicy in the model's predictions.")
print(f"\n A R^2 of {np.mean(tree_r2)} suggests {np.mean(tree_r2)*100} % of the total variance
of the target is predicted by the model. This is very high, and nearly all of the variance in UPDRS
is explained by the model")
print("\n The regession tree performs better than linear regression in all metrics examined.")

#4.)
nn_model = MLPRegressor(random_state=42, max_iter=500)
nn_results = cross_validate(nn_model, x, y, cv=kf, scoring=scoring)

# Convert negative errors to positive (since higher is "better")
nn_mae = -nn_results['test_MAE']
nn_mse = -nn_results['test_MSE']
nn_r2 = nn_results['test_R2']

print("\n \n Neural Network Results ")
print("MAE:", nn_mae)
print("MSE:", nn_mse)
print("R2:", nn_r2)

print("\n Average Results (Neural Network)")
print("Mean MAE:", np.mean(nn_mae))
print("Mean MSE:", np.mean(nn_mse))
print("Mean R2:", np.mean(nn_r2))
```

```
print(f"\n The mean MAE is {np.mean(nn_mae)}, which is a large error, suggesting low
prediction accuracy. This is predicting UPDRS within ~8 units")
print(f"\n The mean MSE is {np.mean(nn_mse)}, which is large, suggesting extreme outliers.
There is not strong consistenicy in the model's predictions.")
print(f"\n A R^2 of {np.mean(nn_r2)} suggests {np.mean(nn_r2)*100} % of the total variance of
the target is predicted by the model. This is very low.")


#5.) comparing all the results

results_summary = pd.DataFrame({
    "Model": ["Linear Regression", "Regression Tree", "Neural Network"],
    "MAE": [np.mean(lr_mae), np.mean(tree_mae), np.mean(nn_mae)],
    "MSE": [np.mean(lr_mse), np.mean(tree_mse), np.mean(nn_mse)],
    "R2": [np.mean(lr_r2), np.mean(tree_r2), np.mean(nn_r2)]
})

print(results_summary)

print("\n Based off the summary table, we can safely conclude that Regression Tree is the best
mocdel for this data set and objective.")
print("\n It produces the lowest MAE and MSE and the highest R^2.")
print("\n The Linear Regression model marginally out performs the neural network model but
ultimtley the results are about the same, and signficantly less accurate than the Regression
Tree.")
print("\n One way to potentially improve the Regression Tree is to limit how deep or complex
the tree can go. With the current implmentation, there is the risk that the model is overfitting
the data.")

#6.) The regression tree is performaing very well and I doubt I will have much imporvement so I
will focus on the neural network. Optimizing the neural network through scaling.

nn_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('nn', MLPRegressor(max_iter=1000, random_state=42))
])

param_grid_nn = {
    'nn__hidden_layer_sizes': [(20,), (20,20)],
    'nn__activation': ['relu'],
    'nn__alpha': [0.001],
    'nn__learning_rate': ['adaptive']
}
```

```
grid_search_nn = GridSearchCV(
    nn_pipeline,
    param_grid_nn,
    cv=kf,
    scoring='neg_mean_absolute_error',
    n_jobs=-1
)

grid_search_nn.fit(x, y)

best_nn_model = grid_search_nn.best_estimator_

nn_opt_results = cross_validate(best_nn_model, x, y, cv=kf, scoring=scoring)

nn_opt_mae = -nn_opt_results['test_MAE']
nn_opt_mse = -nn_opt_results['test_MSE']
nn_opt_r2 = nn_opt_results['test_R2']

print("\n \n Optimized Neural Network Results ")
print("MAE:", nn_opt_mae)
print("MSE:", nn_opt_mse)
print("R2:", nn_opt_r2)

print("\n Average Results (Optimized Neural Network)")
print("Mean MAE:", np.mean(nn_opt_mae))
print("Mean MSE:", np.mean(nn_opt_mse))
print("Mean R2:", np.mean(nn_opt_r2))

results_summary = pd.DataFrame({
    "Model": ["Linear Regression", "Regression Tree", "Neural Network", "Optimized NN"],
    "MAE": [np.mean(lr_mae), np.mean(tree_mae), np.mean(nn_mae), np.mean(nn_opt_mae)],
    "MSE": [np.mean(lr_mse), np.mean(tree_mse), np.mean(nn_mse), np.mean(nn_opt_mse)],
    "R2": [np.mean(lr_r2), np.mean(tree_r2), np.mean(nn_r2), np.mean(nn_opt_r2)]
})
print(results_summary)
print("\n The neural network has imporved, by decreasing Mean MAE and MSE while increasing
R^2.")
print("\n There was one warning that popped up while running the optimized neural network -
Maximum iterations (1000) reached and the optimization hasn't converged yet.")
print("I believe a convergence would be reached if I increased the max iterations, however that
significnatly increased run time. ")
```

Dylan Francis DAAN 682
22MAR2026

Terminal:

```
%runfile
C:/Users/dylan/OneDrive/Documents/GRAD_SCHOOL/DAAN_682/HOMEWORK_9/homework9.
py --wdir

There are 5875 rows, and 22 columns in the features dataframe

=======DATASET_INFO======
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5875 entries, 0 to 5874
Data columns (total 22 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   subject#      5875 non-null   int64
 1   age           5875 non-null   int64
 2   sex           5875 non-null   int64
 3   test_time     5875 non-null   float64
 4   motor_UPDRS   5875 non-null   float64
 5   total_UPDRS   5875 non-null   float64
 6   Jitter(%)     5875 non-null   float64
 7   Jitter(Abs)   5875 non-null   float64
 8   Jitter:RAP    5875 non-null   float64
 9   Jitter:PPQ5   5875 non-null   float64
 10  Jitter:DDP    5875 non-null   float64
 11  Shimmer       5875 non-null   float64
 12  Shimmer(dB)   5875 non-null   float64
 13  Shimmer:APQ3  5875 non-null   float64
 14  Shimmer:APQ5  5875 non-null   float64
 15  Shimmer:APQ11 5875 non-null   float64
 16  Shimmer:DDA   5875 non-null   float64
 17  NHR           5875 non-null   float64
 18  HNR           5875 non-null   float64
 19  RPDE          5875 non-null   float64
 20  DFA           5875 non-null   float64
 21  PPE           5875 non-null   float64
dtypes: float64(19), int64(3)
memory usage: 1009.9 KB
None

=======SUMMARY_STATS======
        subject#         age  ...         DFA         PPE
count  5875.000000  5875.000000  ...  5875.000000  5875.000000
mean     21.494128    64.804936  ...     0.653240     0.219589
```

```
std      12.372279     8.821524  ...     0.070902     0.091498
min       1.000000    36.000000  ...     0.514040     0.021983
25%      10.000000    58.000000  ...     0.596180     0.156340
50%      22.000000    65.000000  ...     0.643600     0.205500
75%      33.000000    72.000000  ...     0.711335     0.264490
max      42.000000    85.000000  ...     0.865600     0.731730

[8 rows x 22 columns]

===== MISSING VALUES =====
subject#        0
age             0
sex             0
test_time       0
motor_UPDRS     0
total_UPDRS     0
Jitter(%)       0
Jitter(Abs)     0
Jitter:RAP      0
Jitter:PPQ5     0
Jitter:DDP      0
Shimmer         0
Shimmer(dB)     0
Shimmer:APQ3    0
Shimmer:APQ5    0
Shimmer:APQ11   0
Shimmer:DDA     0
NHR             0
HNR             0
RPDE            0
DFA             0
PPE             0
dtype: int64

===== DUPLICATE ROWS =====
The total number of duplicate rows is: 0
     age  sex  test_time  total_UPDRS  ...     HNR     RPDE      DFA      PPE
0     72    0     5.6431       34.398  ...  21.640  0.41888  0.54842  0.16006
1     72    0    12.6660       34.894  ...  27.183  0.43493  0.56477  0.10810
2     72    0    19.6810       35.389  ...  23.047  0.46222  0.54405  0.21014
3     72    0    25.6470       35.810  ...  24.445  0.48730  0.57794  0.33277
4     72    0    33.6420       36.375  ...  26.126  0.47188  0.56122  0.19361
  ...  ...       ...          ...  ...     ...      ...      ...      ...
5870   61    0   142.7900       33.485  ...  22.369  0.64215  0.55314  0.21367
```

```
5871   61    0   149.8400      32.988  ...  22.886  0.52598  0.56518  0.12621
5872   61    0   156.8200      32.495  ...  25.065  0.47792  0.57888  0.14157
5873   61    0   163.7300      32.007  ...  24.422  0.56865  0.56327  0.14204
5874   61    0   170.7300      31.513  ...  23.259  0.58608  0.57077  0.15336

[5875 rows x 20 columns]
```

There are 5875 rows, and 20 columns in the features dataframe

```
=======DATASET_INFO======
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5875 entries, 0 to 5874
Data columns (total 20 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   age           5875 non-null   int64
 1   sex           5875 non-null   int64
 2   test_time     5875 non-null   float64
 3   total_UPDRS   5875 non-null   float64
 4   Jitter(%)     5875 non-null   float64
 5   Jitter(Abs)   5875 non-null   float64
 6   Jitter:RAP    5875 non-null   float64
 7   Jitter:PPQ5   5875 non-null   float64
 8   Jitter:DDP    5875 non-null   float64
 9   Shimmer       5875 non-null   float64
 10  Shimmer(dB)   5875 non-null   float64
 11  Shimmer:APQ3  5875 non-null   float64
 12  Shimmer:APQ5  5875 non-null   float64
 13  Shimmer:APQ11 5875 non-null   float64
 14  Shimmer:DDA   5875 non-null   float64
 15  NHR           5875 non-null   float64
 16  HNR           5875 non-null   float64
 17  RPDE          5875 non-null   float64
 18  DFA           5875 non-null   float64
 19  PPE           5875 non-null   float64
dtypes: float64(18), int64(2)
memory usage: 918.1 KB
None

=======SUMMARY_STATS======
             age         sex  ...         DFA         PPE
count  5875.000000  5875.000000  ...  5875.000000  5875.000000
mean     64.804936     0.317787  ...     0.653240     0.219589
std       8.821524     0.465656  ...     0.070902     0.091498
```

```
min     36.000000    0.000000  ...    0.514040    0.021983
25%     58.000000    0.000000  ...    0.596180    0.156340
50%     65.000000    0.000000  ...    0.643600    0.205500
75%     72.000000    1.000000  ...    0.711335    0.264490
max     85.000000    1.000000  ...    0.865600    0.731730

[8 rows x 20 columns]

===== MISSING VALUES =====
age             0
sex             0
test_time       0
total_UPDRS     0
Jitter(%)       0
Jitter(Abs)     0
Jitter:RAP      0
Jitter:PPQ5     0
Jitter:DDP      0
Shimmer         0
Shimmer(dB)     0
Shimmer:APQ3    0
Shimmer:APQ5    0
Shimmer:APQ11   0
Shimmer:DDA     0
NHR             0
HNR             0
RPDE            0
DFA             0
PPE             0
dtype: int64
```

Based on the histogram plot of the target plot, the data is approximately normal, potentiallly slight right skew.
if the data is right skewed, then that would suggest most patients have lower UPDs scores, with fewer high-severity cases.
This grahph shows the features displayed from most positvely correlated to most negatively correlated.

```
 Linear Regression Results
MAE: [8.05379105 7.70138295 8.4283167  8.04474775 8.14023923]
MSE: [ 93.3067207   87.22039122 104.05778523  94.50323596  96.10387592]
R2: [0.1579807  0.20572641 0.13542681 0.17939052 0.171569  ]
```

Here are the average results for Linear Regression

Mean MAE: 8.073695536842244
Mean MSE: 95.0384018064889
Mean R2: 0.17001868860343444

The mean MAE is 8.073695536842244, which is a fairly large error, suggesting low prediction accuracy.

The mean MSE is 95.0384018064889, which means the model is making large prediction errors, suggesting some outliers could exisit.

A R^2 of 0.17001868860343444 suggests only 17.001868860343443 % of the total variance of the target is predicted by the model.

Regression Tree Results
MAE: [0.66437498 0.56804953 0.54679906 0.70058085 0.62551089]
MSE: [8.53895482 4.13721599 6.50632911 7.89802985 6.72398648]
R2: [0.92294269 0.96232439 0.9459416 0.93141824 0.94203815]

Average Results (Regression Tree)
Mean MAE: 0.6210630638297874
Mean MSE: 6.760903249962553
Mean R2: 0.9409330131308664

The mean MAE is 0.6210630638297874, which is a very small error, suggesting high prediction accuracy. This is predicting UPDRS within 1 unit

The mean MSE is 6.760903249962553, which is pretty small, suggesting very few extreme outliers. There is strong consistenicy in the model's predictions.

A R^2 of 0.9409330131308664 suggests 94.09330131308664 % of the total variance of the target is predicted by the model. This is very high, and nearly all of the variance in UPDRS is explained by the model

The regession tree performs better than linear regression in all metrics examined.

Neural Network Results
MAE: [8.17067584 8.07552814 8.79578475 8.26861498 8.48250862]
MSE: [98.19835148 94.27994201 108.90458238 99.32204963 103.7867548]
R2: [0.11383761 0.14143853 0.09515677 0.13754683 0.10534133]

Average Results (Neural Network)
Mean MAE: 8.358622466996184

Dylan Francis DAAN 682
22MAR2026

Mean MSE: 100.89833605986175
Mean R2: 0.11866421381398248

The mean MAE is 8.358622466996184, which is a large error, suggesting low prediction accuracy. This is predicting UPDRS within ~8 units

The mean MSE is 100.89833605986175, which is large, suggesting extreme outliers. There is not strong consistenicy in the model's predictions.

A R^2 of 0.11866421381398248 suggests 11.866421381398249 % of the total variance of the target is predicted by the model. This is very low.

	Model	MAE	MSE	R2
0	Linear Regression	8.073696	95.038402	0.170019
1	Regression Tree	0.621063	6.760903	0.940933
2	Neural Network	8.358622	100.898336	0.118664

Based off the summary table, we can safely conclude that Regression Tree is the best mocdel for this data set and objective.

It produces the lowest MAE and MSE and the highest R^2.

The Linear Regression model marginally out performs the neural network model but ultimtley the results are about the same, and signficantly less accurate than the Regression Tree.

One way to potentially improve the Regression Tree is to limit how deep or complex the tree can go. With the current implmentation, there is the risk that the model is overfitting the data.

```
C:\Users\dylan\anaconda3\Lib\site-
packages\sklearn\neural_network\_multilayer_perceptron.py:781: ConvergenceWarning:
Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't
converged yet.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-
packages\sklearn\neural_network\_multilayer_perceptron.py:781: ConvergenceWarning:
Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't
converged yet.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-
packages\sklearn\neural_network\_multilayer_perceptron.py:781: ConvergenceWarning:
Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't
converged yet.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-
packages\sklearn\neural_network\_multilayer_perceptron.py:781: ConvergenceWarning:
```

```
Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't
converged yet.
  warnings.warn(
C:\Users\dylan\anaconda3\Lib\site-
packages\sklearn\neural_network\_multilayer_perceptron.py:781: ConvergenceWarning:
Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't
converged yet.
  warnings.warn(


 Optimized Neural Network Results
MAE: [4.32994172 4.16428485 4.40969873 4.60539746 4.2482506 ]
MSE: [36.28840059 32.54029136 36.10567776 39.64048454 32.59801533]
R2: [0.67252591 0.70367143 0.70001283 0.65578578 0.71899982]

 Average Results (Optimized Neural Network)
Mean MAE: 4.351514671564978
Mean MSE: 35.434573915238126
Mean R2: 0.6901991528151099
             Model       MAE        MSE       R2
0  Linear Regression  8.073696   95.038402  0.170019
1    Regression Tree  0.621063    6.760903  0.940933
2     Neural Network  8.358622  100.898336  0.118664
3       Optimized NN  4.351515   35.434574  0.690199
```

The neural network has improved, by decreasing Mean MAE and MSE while increasing R^2.

There was one warning that popped up while running the optimized neural network - Maximum iterations (1000) reached and the optimization hasn't converged yet.
I believe a convergence would be reached if I increased the max iterations, however that significantly increased run time.